Loop Engineering is just K8s-style Cybernetics

From Watt's governor to Kubernetes to coding agents

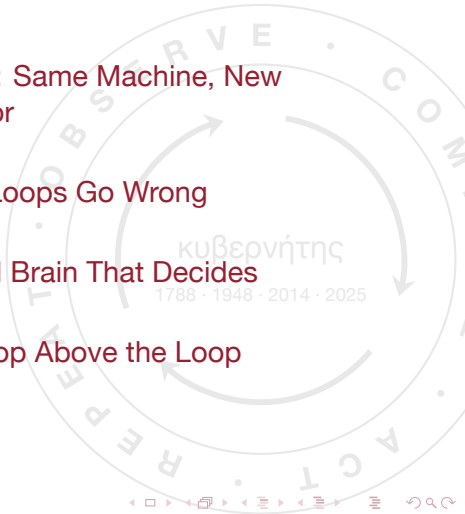
Sayantan Das

MLOps North

November 5, 2026



1. The Steersman
2. Cybernetics in Five Minutes
3. Kubernetes: Cybernetics for Infrastructure
4. Agents: Same Machine, New Actuator
5. When Loops Go Wrong
6. A Small Brain That Decides
7. The Loop Above the Loop



1 The Steersman

2 Cybernetics in Five Minutes

3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



The title says the same thing twice

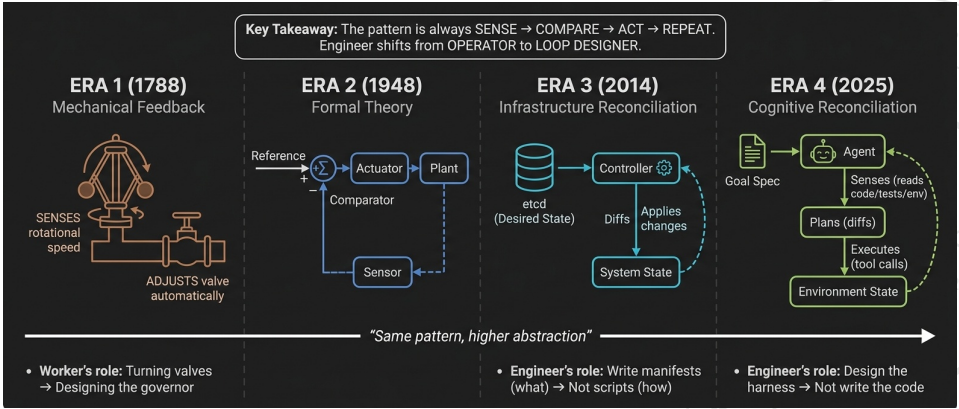
- κυβερνήτης (*kybernētēs*): the steersman of a ship
- **1948**: Norbert Wiener names the science of feedback and control cybernetics after it
- **2014**: Google open-sources a container orchestrator and names it Kubernetes: Greek for helmsman
- **2025**: we put a language model in the loop and call it loop engineering



Claim

This is not an analogy. It is the same machine at a higher level: **state a goal, sense, compare, act, repeat.**

Four eras, one pattern



Each era moves the human **up one level**: from turning the valve to designing the governor.

1 The Steersman

2 Cybernetics in Five Minutes

3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

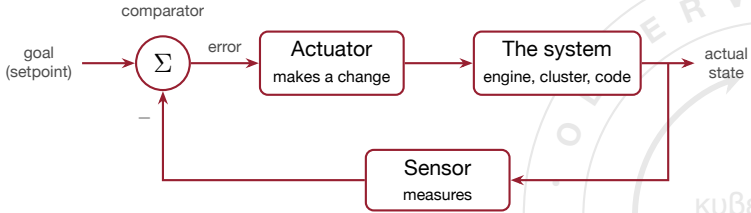
5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



Every feedback loop has the same four parts



	Goal	Sensor	Comparator	Actuator
Watt governor (1788)	target speed	spinning balls	linkage	steam valve
Thermostat	21 °C	thermometer	switch	furnace
Kubernetes (2014)	YAML manifest	API server watch	controller diff	controller writes
Agent loop (2025)	task spec + checks	tests, logs, browser	diff vs. goal	LLM edits code

How control loops fail: Maxwell, *On Governors* (1868)

Governors on steam engines hunted: they overshot, corrected, overshot again. Maxwell's paper on why is one of the founding texts of control theory.

Control term	What it means	In an agent loop
Negative feedback	push against the error	undo a change that made things worse
Damping	slow down after overshooting	smaller changes, wait longer after a miss
Oscillation (hunting)	overshoot, correct, repeat	fix one test, break two, fix those…
Steady-state error	settles short of the goal	stuck: the agent can't close the last gap
Saturation limits	the valve only opens so far	caps on rounds, time and money

Ashby's Law: you can only control what you can sense

Law of Requisite Variety (W. Ross Ashby, *An Introduction to Cybernetics*, 1956)

Only variety can absorb variety: a regulator can only correct the disturbances it can tell apart.

- For an agent loop the **checks are the sensors**. The loop can never be better than its checks.
- A bug the tests can't see is a bug the loop will happily ship.
- Goodhart's law is the dark side: a measure that becomes a target stops being a good measure.
- The cheapest way for an agent to make a failing test pass is to edit the test.

Invest in sensing before you invest in smarter planning.

1 The Steersman

2 Cybernetics in Five Minutes

3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



The reconciliation loop

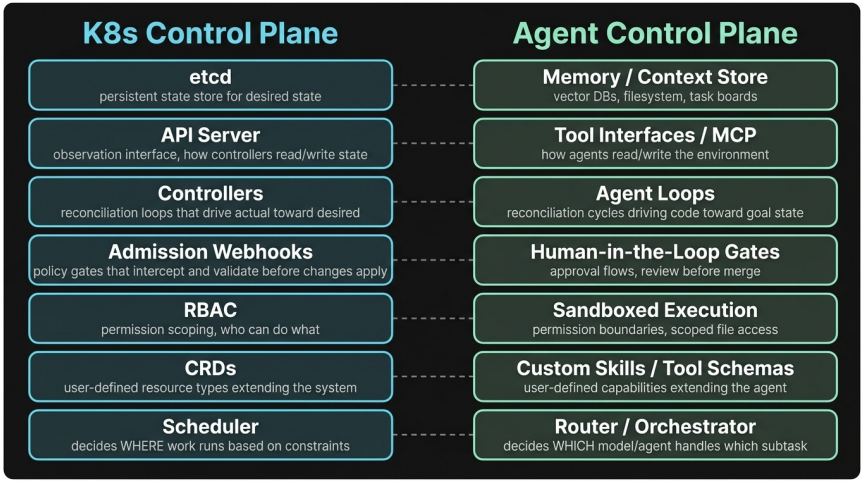
1. **Desired state**: a manifest says what should exist
2. **Observe**: read the current state
3. **Diff**: compare desired vs. actual
4. **Act**: make the smallest change that closes the gap
5. **Repeat**: forever

Level-triggered, not edge-triggered

A controller reacts to *what is true now*, not to the event that just happened. Missed an event? The next pass still sees the gap.

The controller never runs a script from A to B. It only knows how to make the *next correction*. Kill a pod and it comes back: self-healing falls out of the loop. Nobody writes a special case for it.

The control plane, mapped



MLOps already runs on these loops

- **GitOps** (Argo CD, Flux): git holds the desired state, a controller syncs the cluster to it.
- **Continuous training**: a metric drops below the line, retrain until it's back above.
- **Progressive delivery**: canary metrics go bad, roll back automatically.
- **Data validation before training**: a gate that rejects bad input, like an admission webhook.

So what's new?

Only the actuator. The thing making the change is now a language model, and it can make *any* change, including bad ones. Everything else you already know.

1 The Steersman

2 Cybernetics in Five Minutes

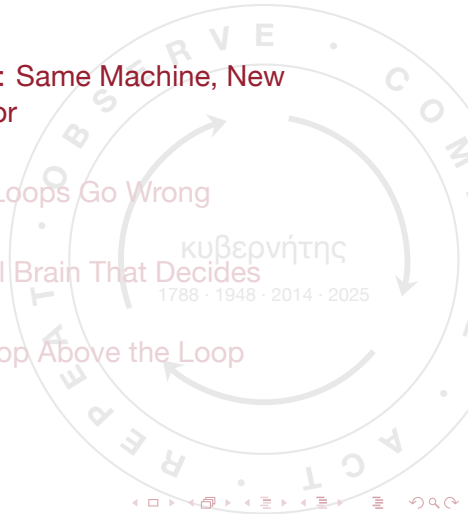
3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

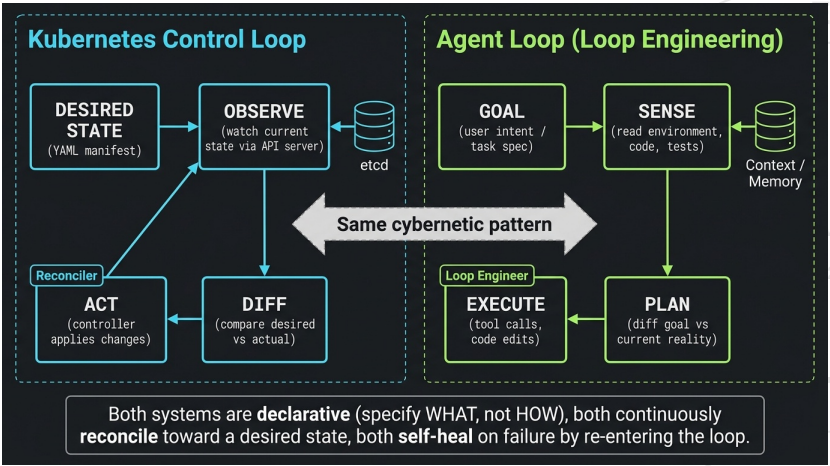
5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



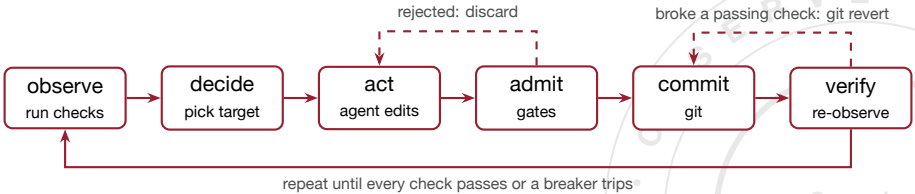
Same loop, new actuator



Design rules that transfer directly

Rule	In Kubernetes	In an agent loop
Declarative	manifest says *what*, not *how*	“all checks pass” beats a 47-step checklist
Observe first	read the API server before acting	read the code and run the tests before editing
Small diffs	change one thing per pass	one target per round, cap the diff size
Idempotent	re-running a pass is harmless	“file contains X”, not “append Y”
Crash-only	controllers can die anytime	progress lives in git, not in the agent’s head
Bounded ownership	labels and selectors	one loop owns one directory

loopctl in one picture



- About 970 lines of Python, standard library only. Git is its etcd: every kept change is a commit.
- The agent is just a shell command. pi, Claude Code, Codex, or a script.
- Kill it at any point; rerun picks up where it left off, with budgets intact.

The goal is a file

```
name  = "churn"
goal  = "Ship-ready churn model: the data contract holds, holdout AUC is at least 0.85,
     and the model card documents the model."
actor = 'pi -p "$(cat)"'      # any agent CLI; the prompt arrives on stdin
fresh_clone = true            # observe a clean clone every round: no stale reads

[[checks]]                    # desired state = every check exits 0
name = "quality"               # (plus "schema" and "model-card")
run  = "python3 eval.py --min-auc 0.85"

[[admission]]                 # policy gate: hands off the sensors
name = "sensors-untouched"
run  = "git diff --cached --quiet -- eval.py validate.py check_card.py schema.json data
    /"

[limits]                      # the safety governor
max_diff_lines   = 80         # damping: big changes are rejected
stall_iterations = 3          # steady-state error: give up
max_regressions  = 2          # oscillation: stop thrashing
```

Every part has a cybernetic name

loopctl	Cybernetics	Kubernetes
`goal` + `[[checks]]`	setpoint	manifest
running the checks	sensor	watch via API server
passing vs. failing	comparator	diff
the agent (`actor`)	actuator	controller writes
`git revert` on regression	negative feedback	rollback
backoff, `max_diff_lines`	damping	rate-limited requeue
`max_regressions`	oscillation guard	—
`stall_iterations`	steady-state error	—
`[[admission]]`, approval	—	admission webhooks
lease file, sandbox	—	RBAC, namespaces
`[decider]`	controller logic	scheduler

Demo 1: a churn model reaches its setpoint (offline)

```
$ ./examples/churn/demo.sh
[churn] observe: 0/3 passing, failing ['schema', 'quality', 'model-card']
[churn] act #1: target `schema`
[churn] committed a0df3d90 (5 lines)
[churn] observe: 1/3 passing, failing ['quality', 'model-card']
[churn] act #2: target `quality`
[churn] committed 37e494e0 (3 lines)
[churn] observe: 1/3 passing, failing ['quality', 'model-card']
[churn] backoff 0.2s
[churn] act #3: target `quality`
[churn] committed f7532e1e (2 lines)
[churn] observe: 2/3 passing, failing ['model-card']
[churn] act #4: target `model-card`
[churn] committed 84efe5aa (22 lines)
[churn] observe: 3/3 passing
[churn] converged
```

Three planted bugs: `contract` not one-hot encoded, train/serve skew, `support_calls` dropped. Round 2 lifts AUC from 0.75 to 0.82: kept, but still under 0.85. Round 3 gets it to 0.874.

Demo 2: a real agent as the actuator

```
actor = 'pi -p "$(cat)"'
```

```
$ ./examples/churn/demo.sh loop.pi.toml
[churn-pi] act #1: target `schema`
[churn-pi] committed de9e9ea8 (5 lines)
[churn-pi] act #2: target `quality`
[churn-pi] committed 9755f658 (5 lines)
[churn-pi] act #3: target `model-card`
[churn-pi] committed 9accb5a8 (28 lines)
[churn-pi] converged
churn-pi: 3 actions, 3 commits, 0 rejected,
  0 reverted, 205.9s acting
```

pi's `quality` round, found from just an AUC number:

```
-NUMERIC = ["tenure_months", "monthly_charges"]
+NUMERIC = ["tenure_months", "monthly_charges",
+           "support_calls"]
-    return sigmoid(sum(w * v for w, v in zip(...x)) + b)
+    xs = scale(x, model["scaler"])
+    return sigmoid(sum(w * v for w, v in zip(...xs)) + b)
```

Both quality bugs in one 5-line change: the dropped feature and the train/serve skew. Holdout AUC 0.874, no gate tripped.

Demo 3: brain outside, hands inside

Each check runs in a throwaway Apple container VM:

```
container run --rm --network none \
  -v "$PWD":/work:ro -w /work \
  python:3.12-slim "$@"
```

```
1 run = "./incvm python3 -m unittest ..."
```

What agent-written code sees inside:

```
home visible: False
network: blocked URLError
write /work: blocked Read-only file system
```

loopctl and the agent stay on the host, so the agent can't rewrite how it's graded. About 1 s per VM; the full loop converged in 55 s.

1 The Steersman

2 Cybernetics in Five Minutes

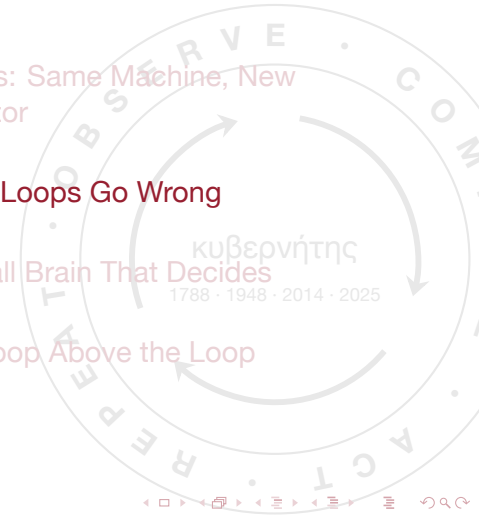
3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



Kubernetes already learned these lessons

Failure	Looks like	loopctl's answer
Thrashing	fix one test, break two	verify every commit; revert on regression; stop after `max_regressions`
Stale reads	acting on an old picture	re-run checks every round, optionally on a fresh clone
Endless loop	goal can't be reached	caps on rounds and time; `stall_iterations`
Fighting loops	two loops, one directory	lease file; edits outside the directory halt the loop
Gaming the sensor	agent edits the test	admission checks reject the change
Blind loop	no signal to steer by	nothing to fix in the loop: add checks

Live failure: thrashing gets caught

A worker that fixes `add` but breaks `mul` every time:

```
[calc] observe: 1/4 passing, failing ['add', 'sub', 'div']
[calc] act #1: target `add`
[calc] committed 84dcb427 (4 lines)
[calc] observe: 1/4 passing, failing ['sub', 'mul', 'div']
[calc] reverted 84dcb427: broke ['mul']
[calc] act #2: target `add`
[calc] committed 8a047e50 (4 lines)
[calc] observe: 1/4 passing, failing ['sub', 'mul', 'div']
[calc] reverted 8a047e50: broke ['mul']
[calc] thrashing: 2 reverted regressions
```

Negative feedback (revert), then a hard stop instead of hunting forever. Every attempt stays in git history for a human to read.

Live failure: the agent tries to move the goalposts

A worker that “fixes” `quality` by lowering the AUC bar in `eval.py`:

```
[churn] act #2: target `quality`
[churn] rejected: admission[sensors-untouched]: exit 1
[churn] act #3: target `quality`
[churn] rejected: admission[sensors-untouched]: exit 1
[churn] act #4: target `quality`
[churn] rejected: admission[sensors-untouched]: exit 1
[churn] unreachable: no progress in 3 iterations; still failing ['quality', 'model-card']
```

- Goodhart’s law, blocked by an admission webhook: one line of shell.
- Reading the holdout set from model code is blocked the same way (`no-holdout-in-model-code`).
- The loop gives up instead of shipping a fake green build.

1 The Steersman

2 Cybernetics in Five Minutes

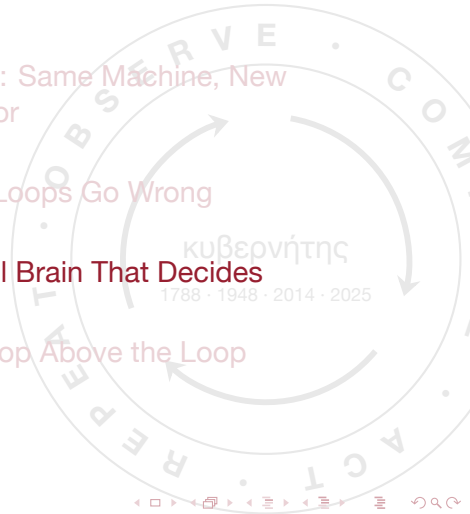
3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



The scheduler doesn't run pods. It picks.

Kubernetes scheduler

- filter: which nodes *can* run this pod?
- score: which is best?
- pick one; the kubelet does the work

Same split for agents

- a small, fast model **decides**
- a big, slow model **does**
- the checks **judge**

TypeSafe Jev

Answers typed questions with probabilities instead of free text. One request per round:

- move: act / ask a human / stop
- target: which failing check next

Guardrails: it can't judge pass/fail or pick a passing check; under 50% confidence it asks a human; if it fails, the loop falls back to the simple rule.

Rules vs. Jev: an impossible check listed first

`gpu` runs `nvidia-smi`: no code change can make it pass on a Mac.

Built-in rule (first failing check)

```
act #1: target `gpu`
rejected: no_op: actor made no changes
act #2: target `gpu`
rejected: no_op: actor made no changes
act #3: target `gpu`
rejected: no_op: actor made no changes
unreachable: no progress in 3 iterations
```

0 of 3 fixable checks fixed.

Jev decider

```
decide: act `add` (0.98)
  target add=0.67 div=0.22 sub=0.09 gpu=0.02
committed 22ef9645
decide: act `sub` (0.98)
  target sub=0.90 div=0.10 gpu=0.00
committed 6f7b3f5c
decide: act `div` (0.96)
  target div=1.00 gpu=0.00
committed 3d5f5a34
decide: stop `gpu` (0.97)
  stop=0.99 ask_human=0.01 act=0.00
unreachable: decider chose stop
```

3 of 3 fixed, then stopped by itself. Whole run: about 3 s.

1 The Steersman

2 Cybernetics in Five Minutes

3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



The hill-climbing machine

The same loop, one level up:

1. ship a v0 agent
2. record what it does (traces)
3. mine traces for failure patterns
4. turn failures into checks
5. improve the agent; repeat

The setpoint is now *agent quality*. You are the actuator.

```
$ loopctl report loop.jev.toml
calc-jev: 3 actions, 3 commits, 0 rejected,
  0 reverted, 0.1s acting
last outcome: unreachable (decider chose stop)

checks observed failing:
     4  gpu
     3  div
     2  sub
     1  add

decisions:
     3  decider: act
     1  decider: stop
```

Try it yourself: the churn task

```
git clone https://github.com/ucalyptus/loop-engineering
cd loop-engineering
./examples/churn/demo.sh
./examples/churn/demo.sh loop.pi.toml
```

The first run is offline and takes about 5 seconds; the second uses your agent. You need Python 3.11+ and git, nothing to `pip install`. The repo is private for now: ask me for access.

1. Watch it converge. Why did `quality` take two rounds?
2. Swap in your agent: pi, Claude Code, Codex.
3. Be the adversary: cheat, then find a cheat the gates miss.
4. Add the sensor it's missing: recall, calibration, AUC per segment.
5. Stretch: let Jev decide.

Takeaways

1. **Declare the exit condition, not the path.** Checks are your setpoint.
2. **Your checks are your sensors.** The loop is only as good as what it can see.
3. **Never let the actuator touch the sensor.** Brain outside, hands inside.
4. **Observe before every action.** Don't trust the agent's memory.
5. **One small change per round, verified.** Undo anything that makes it worse.
6. **Keep actions safe to repeat, and state outside the agent.** Then you can kill it anytime.
7. **Install a governor.** Caps on rounds, time and money, plus a human gate.
8. **Let a small model decide and a big model do.** The checks still judge.

This isn't new. It's Kubernetes for thinking, and Watt's governor for software.

Questions?

Questions?

Goal = checks. Checks = sensors. Keep the judge outside the loop.

Thanks! Let's talk

Thanks!

Sayantan Das
sdas.codes@gmail.com
github.com/ucalyptus
blog.ucalyptus.me

Come find me for:

- help setting up the repo
- the churn task, or wiring in your own agent
- access to `loop-engineering` (private for now)

The blog post this talk is based on

Further reading

- S. Das, *Loop Engineering Is Just K8s-Style Orchestration* (blog.ucalyptus.me)
- N. Wiener, *Cybernetics: Or Control and Communication in the Animal and the Machine*, 1948
- W. R. Ashby, *An Introduction to Cybernetics*, 1956
- J. C. Maxwell, *On Governors*, Proceedings of the Royal Society, 1868
- controller-runtime and the Kubernetes scheduler docs
- Apple container and pi-cvm (github.com/ucalyptus/pi-cvm)
- TypeSafe docs (docs.typesafe.ai)